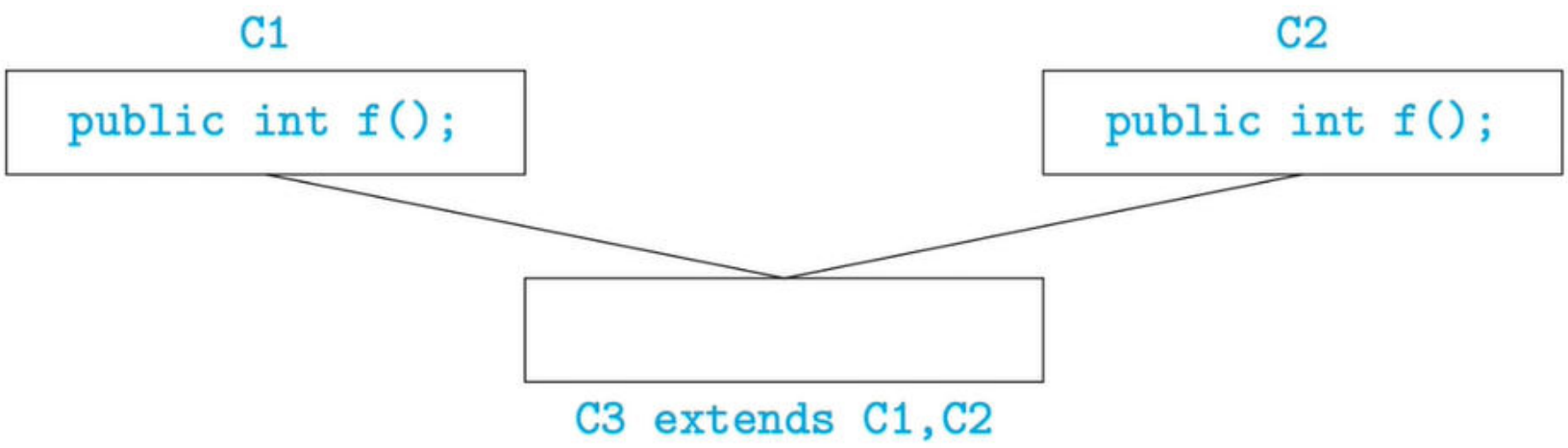
The Java class hierarchy

Type	Lecture
Date	@January 8, 2022
Lecture #	4
Lecture URL	https://youtu.be/WRN3QWICaag
Notion URL	https://21f1003586.notion.site/The-Java-class-hierarchy-42f67eff075e438a81fe1dc7c31725fa
Week #	3

Multiple inheritance



- Can a sub-class extend multiple parent classes?
- If `f()` is not overridden, which `f()` do we use in `C3`?
- Java does NOT allow multiple inheritance
- C++ allows this only if `C1` and `C2` have no conflict

Java class hierarchy

- No multiple inheritance — tree-like
- In fact, there is a universal superclass `Object`
- Useful methods defined in `Object`

```
public boolean equals(Object o)  // defaults to pointer equality
```

```
public String toString()  // converts the value of the instance variables to String
```

- For Java objects `x` and `y`, `x == y` invokes `x.equals(y)`
- To print `o`, use `System.out.println(o + "");`
  - Implicitly invokes `o.toString()`
- Can exploit the tree structure to write generic functions
  - Example: search for an element in an array

```
public int find(Object[] objarr, Object o) {
  int i;
  for(i = 0; i < objarr.length(); ++i) {
    if(objarr[i] == o) {
      return i;
    }
  }
  return -1;
}
```

- Recall that `==` is pointer equality, by default
- If a class overrides `equals()`, dynamic dispatch will use the redefined function instead of `Object.equals()` for `objarr[i] == o`

Overriding functions

- For instance, a class `Date` with instance variables `day`, `month` and `year`
- May wish to override `equals()` to compare the object state, as follows

```
public boolean equals(Date d) {
  return this.day == d.day && this.month == d.month && this.year == d.year;
}
```

- Unfortunately, `boolean equals(Date d)` does not override `boolean equals(Object o)`
- Should write this instead

```
public boolean equals(Object d) {
  if(d instanceof Date) {
    Date myd = (Date) d;
    return this.day == myd.day && this.month == myd.month && this.year == myd.year;
  }

  return false;
}
```

- Overriding looks for "closest" match
- Suppose we have `public boolean equals(Employee e)` but no `equals()` in `Manager`
- Consider

  `Manager m1 = new Manager(...);`

  `Manager m2 = new Manager(...);`

  `...`

  `if(m1.equals()m2) { ... }`
- `public boolean equals(Manager m)` is compatible with both `boolean equals(Employee e)` and `boolean equals(Object o)`
- Use `boolean equals(Employee e)`

Summary

- Java does not allow multiple inheritance
  - A sub-class can only extend one parent class
- The Java class hierarchy forms a tree
- The root of the hierarchy is a built-in class called `Object`
  - `Object` defines default functions like `equals()` and `toString()`
  - These are implicitly inherited by any class that we write

- When we override functions, we should be careful to check the signature